

Linguaggio Assembly

Cheng Kai Liu

Istituto Superiore E. Fermi

Sistemi E Reti

A.S. 2021/2022 - 3CIIN

Introduzione

Cos'è il linguaggio Assembly

Il linguaggio Assembly è un linguaggio di programmazione a basso livello specifico per un'architettura di computer, a differenza della maggior parte dei linguaggi ad alto livello, che possono essere utilizzati su più sistemi. Infatti, tra diverse architetture, il linguaggio Assembly cambia.

Cosa bisogna conoscere prima di cominciare a programmare in Assembly

È necessario conoscere i registri della CPU e quali operazioni il processore è in grado di compiere. Inoltre bisogna sapere come funziona la RAM, quindi come sono salvati i dati e gli indirizzi della memoria.

Come è possibile che un circuito elettronico riesca ad eseguire istruzioni software

Un circuito è in grado di eseguire istruzioni software grazie alla traduzione del codice da Assembly in bit, cioè valori logici vero (1) e falso (0). Il codice è convertito dall'Assembly al linguaggio macchina attraverso l'*Assembler*. Quest'ultimo prende il codice e lo traduce in bit che verranno usati come input nella CPU, che attraverso le sue porte logiche fa i calcoli necessari per eseguire l'operazione richiesta.

Come viene usato oggi il linguaggio Assembly

Il linguaggio Assembly viene usato principalmente per scrivere i codici per compilatori per linguaggi di programmazione ad alto livello oppure per nuovi dispositivi elettronici che hanno un'architettura diversa.

HC08

Struttura di un programma Assembly HC08

HC08 è un microcontrollore composto da CPU a 8 bit, ROM, RAM 128 byte, I/O e periferiche. La struttura del codice è formato da:

- **Costanti:** definiscono gli indirizzi di memoria da cui parte il programma a salvare le variabili;
- **Dati:** variabili e procedure del programma;
- **Programma:** raggruppa le istruzioni operative;
- **Reset:** raggruppa le istruzioni per azzerare il programma.

Come è possibile eseguire un programma per la CPU HC08

È possibile eseguire un programma in Assembly HC08 attraverso emulatori o codici JavaScript implementati in siti web, come <https://www.docsalvi.it/hc08-online/>

Parti fondamentali di un'istruzione Assembly HC08

Le istruzioni in Assembly sono composte da **Label**, **Op. Code**, **Operands**:

- **Label** (etichetta): sono identificatori che vengono sostituite dall'assemblatore con l'indirizzo dell'istruzione in memoria, utili per fare i *jump* da un'istruzione all'altra; vengono utilizzate perché permettono di trovare più facilmente un punto del programma e ne facilitano la modifica;
- **Operation Code** (codice operativo): è l'operazione che la CPU deve eseguire;
- **Operandi:** sono le variabili, gli indirizzi o i numeri che vengono processate con l'istruzione in Op. Code. In alcune istruzioni possono anche non esserci.

Istruzioni più frequenti

- **MOV:** assegna a una variabile un valore numerico o un valore contenuto in un indirizzo in memoria o contenuto in una variabile. Il valore originario viene preceduto da # e seguito da ','.
Es. `MOV #10, var1` → assegna alla variabile `var1` il valore 10

MOV #var2, var1 → assegna alla variabile **var1** il valore in **var2**

- **LDA**: carica nel registro accumulatore A il valore memorizzato in RAM o un valore numerico seguito dal #

Es. **LDA #10** → carica in A il valore numerico 10

LDA #var → carica in A il valore presente nella variabile **var**

Esempio di codice

MAIN:

MOV #5,num1 → carica 5 in num1

MOV #1,count → carica 1 in count

MOV #0,risultato → carica 0 in risultato

LDA count → carica in A il valore contenuto in count

CICLOW: → inizio ciclo CICLOW

CMP num1 → confronta num1 con il valore in A

BGT FINE → se $A > \text{num1}$ salta a FINE

LDA risultato → carica in A il risultato

ADD num1 → aggiunge ad A il valore in num1

STA risultato → carica il contenuto in A nella variabile risultato

LDA count → carica in A il valore in count

ADD #1 → aggiunge 1 al valore in A

STA count → carica il contenuto in A nella variabile count

BRA CICLOW → salta all'inizio di CICLOW

FINE: → inizio ciclo FINE

BRA FINE → salto incondizionato a FINE (loop per terminare il programma)

Conclusioni

Cosa è possibile fare con l'Assembly

In assembly è possibile programmare codici a basso livello per specifici sistemi, con il loro linguaggio Assembly progettato appositamente per la loro architettura, si possono creare programmi molto veloci.

Vantaggi di Assembly

È possibile sfruttare a pieno l'hardware, con l'utilizzo di tutte le istruzioni hardware. Il codice sviluppato non necessita modifiche perché lo standard per una certa architettura rimane invariabile.

Svantaggi di Assembly

I codici in Assembly funzionano per solo un certo tipo di architettura e la sintassi è molto complicata.

Divisione

```

STARTPGM    = 0xee00          ; L'inizio dell'ipotetica ROM
RESETVECT   = 0xffff          ; La posizione del vettore di reset
STARTDATI    = 0b10000000     ; Inizio area dati
; le pseudo istruzioni sul come tradurre sono precedute dal simbolo punto

        .area    DATI (ABS)    ; tutti i .area DATI verranno messi insieme
        .org     STARTDATI     ; indirizzo da cui parte l'area di definizione
DATI
;
dividendo: .blkb 1
divisore:  .blkb 1
ris:       .blkb 1
;

        .area    PROGRAMMA (ABS)
        .org     STARTPGM      ;inizio area PROGRAMMA
MAIN:    MOV #10, divisore      ; variabili
        MOV #3, dividendo
        MOV #0, ris
LOOP:
        LDA divisore           ; carica divisore nel registro A
        CMP dividendo
        BLT FINE ; se divisore < dividendo termina
        ; else
        SUB dividendo          ; sottrae dividendo al divisore
        STA divisore           ; salva dividendo sottratto
        LDA ris                ; risultato in A
        ADD #1                 ; ris=ris+1
        STA ris                ; salva risultato
        LDA divisore
        CMP dividendo
        BRA LOOP               ; salta al LOOP
;
FINE:    BRA FINE               ; Loop infinito per fermare il programma
;

        .area    RESET (ABS)
        .org     RESETVECT     ;inizio area RESET
        .word    MAIN           ; alloca 2 byte con valore MAIN

```

Codice	Registri
ee00 6e 0a 81 MOV #0a,\$81	Accumulatore..... : 01 (1)
ee03 6e 03 80 MOV #03,\$80	H:X..... : 0000
ee06 6e 00 82 MOV #00,\$82	Stack Pointer..... : 00ff
ee09 c6 00 81 LDA #0081	Program Counter..... : ee27
ee0c c1 00 80 CMP #0080	Condition Code Register : v11hINxC
ee0f 91 16 BLT ee27	
ee11 c0 00 80 SUB #0080	Dati
ee14 c7 00 81 STA #0081	0080 03 01 03 ff ff ff ff ff
ee17 c6 00 82 LDA #0082	0088 ff ff ff ff ff ff ff ff
ee1a ab 01 ADD #01	0090 ff ff ff ff ff ff ff ff
ee1c c7 00 82 STA #0082	0098 ff ff ff ff ff ff ff ff
ee1f c6 00 81 LDA #0081	00a0 ff ff ff ff ff ff ff ff
ee22 c1 00 80 CMP #0080	00a8 ff ff ff ff ff ff ff ff
ee25 20 e2 BRA ee09	00b0 ff ff ff ff ff ff ff ff
ee27 20 fe BRA ee27	00b8 ff ff ff ff ff ff ff ff
ee29 ff STX ,X	00c0 ff ff ff ff ff ff ff ff
	00c8 ff ff ff ff ff ff ff ff
	00d0 ff ff ff ff ff ff ff ff
Comandi	I/O
s - step	s 0 0
r - reset	12345678901234567890
t - Commuta tasto	Tasto: ☐
f - Ridisegna display	
q - esci	

Alla fine del programma, dopo aver calcolato 10/3

AREA DATI:

indirizzo 0080 → 03 (divisore)
 01 (resto)
 03 (risultato)